

Fast AVX-512 Implementation of the Optimal Ate Pairing on BLS12-381

Hao Cheng, Georgios Fotiadis, Johann Großschädl, Daniel Page

上次内容回顾

在上次分享中，我们介绍了 Weil pairing 和 Tate pairing 在密码学中的应用。其中对于 reduced Tate pairing 的内容摘要如下：

Tate pairing 与 weil pairing 的区别是其椭圆曲线定义在有限域 $\mathbb{F}_q$ 上，而后者可以是任意域。我们定义：

$$\hat{\tau}_\ell(P, Q) = \left(\frac{f_{\ell,P}(Q+S)}{f_{\ell,P}(S)} \right)^{(q-1)/\ell} \in \mathbb{F}_q$$

其中 ℓ 是质数， $P \in E(\mathbb{F}_q)[\ell]$ ， $Q \in E(\mathbb{F}_q)$ ， $q \equiv 1 \pmod{\ell}$ 。

实际上更一般的情况是，reduced Tate pairing 也可以在扩域 $\mathbb{F}_{q^k}$ 定义，这里的 k 称为嵌入次数（embedding degree），满足 $\ell \mid q^k - 1$ 。则上一节的计算形式则应该对应：

$$\hat{t}_\ell(P, Q) = \left(\frac{f_{\ell,P}(Q+S)}{f_{\ell,P}(S)} \right)^{(q^k-1)/\ell} \in \mu_\ell \subset \mathbb{F}_{q^k}^*.$$

在密码学语境中，原来的分式 $\frac{f_{\ell,P}(Q+S)}{f_{\ell,P}(S)}$ 通常会被简写为 $f_{\ell,P}(D_Q)$ ，其中 D_Q 是代表 Q 的零次除子，例如 $D_Q = [Q+S] - [S]$ 。其实就是上次分享中“有理函数对除子求值”的形式：正项进分子，负项进分母。

从 reduced Tate 到 optimal ate

然后回到上一次组会的未来预告：

可以发现，reduced Tate pairing 的 Miller function 计算量比 Weil pairing 少一半，并且仍满足单位根和双线性两个性质，所以在密码学中更受青睐。例如 Tate pairing 的变种 ate pairing 在以太坊中被广泛使用，而 CHES 上现在仍有针对 optimal ate pairing 的优化工作。

<https://eprint.iacr.org/2025/1283>

上一次分享提到：Weil pairing 需要两个 Miller 函数；reduced Tate 只需要一个。接下来真正的优化，是让这一个 Miller loop 尽可能短。

Miller 算法按整数 m 的二进制展开做 double-and-add：

$$\operatorname{div}(f_P) = m[P] - [mP] - (m - 1)[\mathcal{O}]$$

这里的 m 就是 Miller loop 的参数； m 越短，循环越短。

- reduced Tate pairing：Miller loop 扫 ℓ ，其中 ℓ 是曲线大素数阶子群的阶。
- ate pairing：针对椭圆曲线，Miller loop 扫 $T = t - 1$ ，其中 t 是 Frobenius trace。
- optimal ate pairing：进一步利用 pairing-friendly curve 的参数化结构，让 loop parameter 接近理论最短。

BLS12-381 实现

这里以 BLS12-381 为例。它是一条常用的 pairing-friendly elliptic curve，定义在约 381-bit 的素域上，嵌入次数 $k = 12$ ，常用于 BLS signatures、zk-SNARKs 和以太坊相关协议。

回到上一页，对于经典的 reduced Tate pairing，Miller loop 参数

$$\ell \approx 255\text{-bit}$$

对于 ate pairing，Hasse 定理

$$\#E(\mathbb{F}_q) = q + 1 - t, \quad |t| \leq 2\sqrt{q}$$

告诉我们 $t \sim \sqrt{q}$ ，因此 Miller loop 参数 $T = t - 1 < 192\text{-bit}$ 。

而对于 optimal ate pairing，通过进一步利用曲线 seed z ，把 loop parameter 降到：

$$z \approx 64\text{-bit}.$$

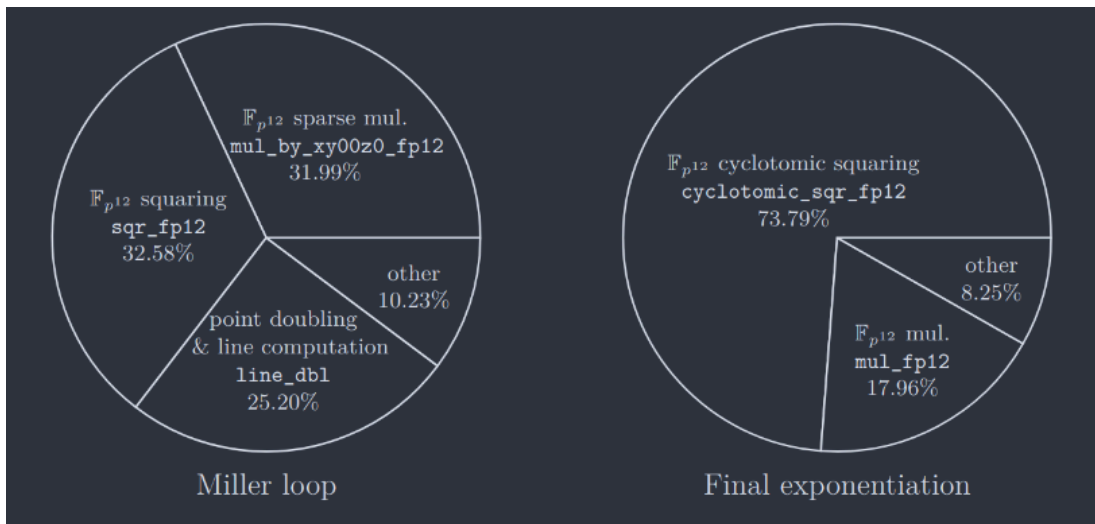
因此可以粗略理解为：optimal ate 用更短的整数来跑 Miller loop，相比于最开始的 reduced Tate pairing 有 $4\times$ 的速度提升。

这篇论文优化什么？

我们现在要面对的是这样的一个式子，我们记 optimal ate pairing 为 e_{OATE} ，则：

$$\hat{\tau}_\ell(P, Q) = f_{\ell, P}(D_Q)^{(q^k-1)/\ell} \implies e_{\text{OATE}}(Q, P) = f_{z, Q}(D_P)^{(p^{12}-1)/r}.$$

主要分为两部分的计算，第一部分就是 $f_{z, Q}(D_P)$ 这个 Miller loop，第二部分就是做 $(p^{12} - 1)/r$ 这个 final exponentiation，下图给出了这两块各自时间的占比（前者 42%，后者 57%）：



然后我们立即有两个问题需要解答：What to optimize? How to optimize?

对于第一个问题，为了得到最大 speedup，Cheng 等人自然是除了 other 之外，全都要优化。具体而言，论文提出了以下六个优化点：

- $\mathbb{F}_{p^{12}}$ cyclotomic squaring (`cyclotomic_sqr_fp12`)
- $\mathbb{F}_{p^{12}}$ multiplication (`mul_fp12`)
- $\mathbb{F}_{p^{12}}$ sparse multiplication (`mul_by_xy00z0_fp12`)
- $\mathbb{F}_{p^{12}}$ squaring (`sqr_fp12`)
- point doubling & line computation (`line_dbl`)
- point addition & line computation (`line_add`)

而对于后者，他们敏锐的发现 BLS12-381 曲线广泛使用的高性能实现 blst（就是用在以太坊的那个）的确使用了 x64 assembly、BMI2/ADX 等标量大整数优化；但这并不是 SIMD 的向量化实现，因此这给具有高度向量化的 AVX-512IFMA (Integer Fused Multiply-Add) 指令的机器带来了优化空间。

这其实也解释了为什么 blst 不做，因为支持这种指令集的设备很少，而区块链共识/验证路径需要稳定、可移植，这本身就存在一些 tradeoff.

AVX-512IFMA 本身能力

论文使用的指令集是 AVX512-IFMA，AVX512 代表存在长度为 512bit 的向量寄存器，可以看成 8 个 64-bit lane 的小寄存器同时进行某种相同的操作（比如说 xor、and 等等）；IFMA 则代表一种专门的 SIMD 操作：52-bit 多精度整数乘法。

具体而言，AVX512-IFMA 有两个指令：vpmadd52luq 和 vpmadd52huq，它们每个 lane 做的事情可以粗略理解为：

```
dst += low_52_bits(a * b)
dst += high_52_bits(a * b)
```

也就是取每个 lane 64 bit 的低 52 bit，做乘法操作会得到 104 bit，将高 52 bit 和低 52 bit 直接相加得到结果。

而 AVX512 有 8 个 lane，也就是一条指令可以同时做 8 次这样的操作。

这个指令作为论文的 building block，用于优化前面的所有六个 primitive.

但实际上，论文并没有完整用满这 52-bit，而是每个 lane 仅用了 48-bit，而 $48 \times 8 = 384 > 381$ ，刚好能装下一个 field element。另一个好处是，如果一个 lane 装满 52-bit，对这个 lane 进行加法操作时有可能超过 IFMA 乘法允许的 52-bit 输入范围，在下次乘法前需要把进位传给更高位的 limb，反而损失性能。

Case Study: $\mathbb{F}_{p^{12}}$ cyclotomic squaring

$$f \in \mathbb{F}_{p^{12}}^*$$

$$f^{(p^{12}-1)/r} = \left(f^{(p^6-1)(p^2+1)} \right)^{(p^4-p^2+1)/r}$$

easy part

$$g = f^{(p^6-1)(p^2+1)}$$

$$g \in G_{\Phi_6}(\mathbb{F}_{p^2}), \quad g^{p^4-p^2+1} = 1$$

hard part: 反复平方

$$g = a + bw + cw^2, \quad a, b, c \in \mathbb{F}_{p^4}$$

$$g^2 = A + Bw + Cw^2$$

$$A = 3a^2 - 2\bar{a}$$

$$B = 3c^2v + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}$$

Granger-Scott

$$\text{主要成本} = a^2 + b^2 + c^2 = 3S_4 = 6S_2 + 3M_2$$

three $\mathbb{F}_{p^4}$ squarings

$$\begin{array}{ll} (B, C) : & (2 \times 2 \times 2 \times 1)\text{-way} \\ & (2 \times 1 \times 4 \times 1)\text{-way} \\ A : & (1 \times 2 \times 2 \times 2)\text{-way} \\ & (1 \times 1 \times 4 \times 2)\text{-way} \end{array}$$

hybrid vectorization

$$x \in \mathbb{F}_p : \quad x = \sum_{i=0}^7 x_i 2^{48i}$$

vpmadd52luq/huq

IFMA / 48-bit limb

Easy part

$$f \in \mathbb{F}_{p^{12}}^*$$

$$f^{(p^{12}-1)/r} = \left(f^{(p^6-1)(p^2+1)} \right)^{(p^4-p^2+1)/r}$$

easy part

$$g = f^{(p^6-1)(p^2+1)}$$

$$g \in G_{\Phi_6}(\mathbb{F}_{p^2}), \quad g^{p^4-p^2+1} = 1$$

hard part: 反复平方

$$g = a + bw + cw^2, \quad a, b, c \in \mathbb{F}_{p^4}$$

$$g^2 = A + Bw + Cw^2$$

$$A = 3a^2 - 2\bar{a}$$

$$B = 3c^2v + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}$$

Granger-Scott

$$\text{主要成本} = a^2 + b^2 + c^2 = 3S_4 = 6S_2 + 3M_2$$

three $\mathbb{F}_{p^4}$ squarings

$$(B, C) : \quad (2 \times 2 \times 2 \times 1)\text{-way}$$

$$(2 \times 1 \times 4 \times 1)\text{-way}$$

$$A : \quad (1 \times 2 \times 2 \times 2)\text{-way}$$

$$(1 \times 1 \times 4 \times 2)\text{-way}$$

hybrid vectorization

$$x \in \mathbb{F}_p : \quad x = \sum_{i=0}^7 x_i 2^{48i}$$

vpmadd521uq/huq

IFMA / 48-bit limb

根据 Miller loop 的定义，其输出 $f \in \mathbb{F}_{p^{12}}^*$ ，并且我们需要计算 $f^{(p^{12}-1)/r}$ 。 $p^{12} - 1$ 的因式分解为：

$$p^{12} - 1 = (p^6 + 1)(p^6 - 1) = (p^2 + 1)(p^4 - p^2 + 1)(p^6 - 1)$$

所以：

$$\frac{p^{12} - 1}{r} = (p^6 - 1)(p^2 + 1)\frac{p^4 - p^2 + 1}{r}$$

对于前两者，也就是计算 f^{p^6-1} 或者 f^{p^2+1} ，Frobenius automorphism 告诉我们 $(a + b)^{p^k} = a^{p^k} + b^{p^k}$ ，因此乘方的复杂度相比直接展开更低。

具体而言，我们先计算 $h = f^{p^6-1} = f^{p^6} f^{-1}$ ，后者涉及到一次比较昂贵的 $\mathbb{F}_{p^{12}}$ 求逆。然后再做 $g = h^{p^2} h$ ，即可得到 $g = f^{(p^6-1)(p^2+1)}$ 的结果。总之我们仅仅用了**两次** Frobenius，**两次乘法**和**一次求逆**就完成了前两者操作。（论文中的 easy part）

同时，由于 $\text{ord}(g) \mid (p^4 - p^2 + 1)$ ，同时我们知道 $\Phi_6(X) = X^2 - X + 1$ ，令 $X = p^2$ ，便容易理解 $g \in G_{\Phi_6}(\mathbb{F}_{p^2})$ ，也就是接下来的运算是在分圆子群进行的。（论文中的 hard part）

Hard part

$$f \in \mathbb{F}_{p^{12}}^*$$

$$f^{(p^{12}-1)/r} = \left(f^{(p^6-1)(p^2+1)} \right)^{(p^4-p^2+1)/r}$$

easy part

$$g = f^{(p^6-1)(p^2+1)}$$

$$g \in G_{\Phi_6}(\mathbb{F}_{p^2}), \quad g^{p^4-p^2+1} = 1$$

hard part: 反复平方

$$g = a + bw + cw^2, \quad a, b, c \in \mathbb{F}_{p^4}$$

$$g^2 = A + Bw + Cw^2$$

$$A = 3a^2 - 2\bar{a}$$

$$B = 3c^2v + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}$$

Granger-Scott

$$\text{主要成本} = a^2 + b^2 + c^2 = 3S_4 = 6S_2 + 3M_2$$

three $\mathbb{F}_{p^4}$ squarings

$$(B, C) : \quad \begin{array}{l} (2 \times 2 \times 2 \times 1)\text{-way} \\ (2 \times 1 \times 4 \times 1)\text{-way} \end{array}$$

$$A : \quad \begin{array}{l} (1 \times 2 \times 2 \times 2)\text{-way} \\ (1 \times 1 \times 4 \times 2)\text{-way} \end{array}$$

hybrid vectorization

$$x \in \mathbb{F}_p : \quad x = \sum_{i=0}^7 x_i 2^{48i}$$

$$\text{vpmadd52luq/huq}$$

IFMA / 48-bit limb

针对 hard part 的第一个问题，就是 r 的值是不是任意的一个能整除 $p^4 - p^2 + 1$ 的一个整数？答案是否定的。

事实上，我们将要计算的式子 g^E ， $E = \frac{p^4 - p^2 + 1}{r}$ 中的 p 和 r 都是前文中提到的 BLS12-381 曲线的 seed z 生成， z 的具体值为 `-0xd2010000000010000`。具体而言， p 和 r 满足：

$$r = r(z) = z^4 - z^2 + 1 = \Phi_{12}(z) \quad p = p(z) = \frac{(z-1)^2 r}{3} + z$$

通过代数恒等变换，我们可以得到 HHT 公式^[1]：

$$3E = (z-1)^2(z+p)(z^2+p^2-1) + 3$$

由于 $z \equiv 1 \pmod{3}$ ，实际上也可以写成 $E = \frac{(z-1)^2}{3}(z+p)(z^2+p^2-1) + 1$ 。但实际上 $\frac{(z-1)^2}{3}$ 的 Hamming weight 过大，因此 HHT 选择直接计算前文 easy part 的结果 g^{3E} ，没有开立方。

注意这里并不影响 pairing 的正确性——由于 $\gcd(3, r) = 1$ ，所以 pairing 映射 $e_{\text{OATE}}(Q, P) \rightarrow e_{\text{OATE}}(Q, P)^3$ 是自同构，仍满足 pairing 相关性质并被密码学正常使用。

1. <https://eprint.iacr.org/2020/875> ↩

总之，最终问题便可转化为对求 $x^z, x^{z^2}, x^p, x^{p^2}$ 以及 x^{-1} 的运算操作。

- 对于 x^p, x^{p^2} ，与前文相同，可以用 Frobenius 较为快速地计算；
- x^{-1} 中的 x 满足 $x^{p^4-p^2+1} = 1$ ，故 $x^{p^6+1} = (x^{p^4-p^2+1})^{p^2+1} = 1$ ，即 $x^{-1} = x^{p^6}$ ，本质上还是 Frobenius。
- 然后就是 x^z, x^{z^2} 了。由于 $z = -(2^{63} + 2^{62} + 2^{60} + 2^{57} + 2^{48} + 2^{16})$ ，是一个负数。我们先计算 $x^{|z|}$ 然后做一下上一步的 inversion 即可。

具体而言：

$$x^{|z|} = x^{2^{63}} x^{2^{62}} x^{2^{60}} x^{2^{57}} x^{2^{48}} x^{2^{16}}$$

也就是做一次 $x^{|z|}$ ，成本是 63 次 cyclotomic squaring 操作和 5 次 multiplication 操作，自然前者就成了论文优化的最重要瓶颈。

这就是为什么要让 z 的 Hamming weight 尽量小，主要是为了减少 multiplication 的操作次数。

为了优化 cyclotomic squaring 本身，也一个高效的公式叫 Granger-Scott 公式，这便是下一个部分的内容。

Granger-Scott

$$f \in \mathbb{F}_{p^{12}}^*$$

$$f^{(p^{12}-1)/r} = \left(f^{(p^6-1)(p^2+1)} \right)^{(p^4-p^2+1)/r}$$

easy part

$$g = f^{(p^6-1)(p^2+1)}$$

$$g \in G_{\Phi_6}(\mathbb{F}_{p^2}), \quad g^{p^4-p^2+1} = 1$$

hard part: 反复平方

$$g = a + bw + cw^2, \quad a, b, c \in \mathbb{F}_{p^4}$$

$$g^2 = A + Bw + Cw^2$$

$$A = 3a^2 - 2\bar{a}$$

$$B = 3c^2v + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}$$

Granger-Scott

$$\text{主要成本} = a^2 + b^2 + c^2 = 3S_4 = 6S_2 + 3M_2$$

three $\mathbb{F}_{p^4}$ squarings

$$(B, C) : \quad (2 \times 2 \times 2 \times 1)\text{-way}$$

$$(2 \times 1 \times 4 \times 1)\text{-way}$$

$$A : \quad (1 \times 2 \times 2 \times 2)\text{-way}$$

$$(1 \times 1 \times 4 \times 2)\text{-way}$$

hybrid vectorization

$$x \in \mathbb{F}_p : \quad x = \sum_{i=0}^7 x_i 2^{48i}$$

vpmadd521uq/huq

IFMA / 48-bit limb

在上一节中，主要耗时的原语是对元素 g 反复做 cyclotomic squaring 操作，即映射：

$$g \mapsto g^2, \quad g \in G_{\Phi_6}(\mathbb{F}_{p^2}) \subset \mathbb{F}_{p^{12}}^*.$$

当我们令 $q = p^2$ [1]，则 $G_{\Phi_6}(\mathbb{F}_{p^2}) = G_{\Phi_6}(\mathbb{F}_q)$ ，后者刚好是 Granger–Scott 原工作的对象 [2]。具体而言，Granger–Scott 公式选择的扩域视角为 $\mathbb{F}_q \subset \mathbb{F}_{q^2} \subset \mathbb{F}_{q^6}$ 。

首先考虑第一个二次扩张，通常的构造方式是 $\mathbb{F}_{q^2} = \mathbb{F}_q[v]/(v^2 - \xi)$ ，这里 $\xi \in \mathbb{F}_q$ 是非二次剩余。这个域的每个元素是形如 $x = a + bv$ 的形式，我们计算平方操作：

$$x^2 = (a + bv)^2 = (a^2 + \xi b^2) + 2abv$$

特别地，计算 p 次方等同于共轭。由二次扩域的性质可得 $v^q = -v$ ，则：

$$x^q = (a + bv)^q = a + bv^q = a - bv = \bar{x}$$

无需乘法或者平方操作。

1. 记住这个等式，接下来会反复用到，我不用再明说 ↩

下一步就是从 $\mathbb{F}_{q^2}$ 跳跃到 $\mathbb{F}_{q^6}$ ，我们构造三次扩域：

$$\mathbb{F}_{q^6} = \mathbb{F}_{q^2}[w]/(w^3 - v)$$

其中 $g = a + bw + cw^2$, $a, b, c \in \mathbb{F}_{q^2}$ ，同时也有 $w^3 = v$ 。

回到我们的目标， $\mathbb{F}_{q^6}$ 的平方操作。先普通展开平方：

$$\begin{aligned} g^2 &= (a + bw + cw^2)^2 \\ &= a^2 + 2abw + (b^2 + 2ac)w^2 + 2bcw^3 + c^2w^4 \end{aligned}$$

利用 $w^3 = v$ 这个关系，可以将 w^3, w^4 规约回 $\{1, w\}$ ，得到形如 $A + Bw + Cw^2$ 的形式，系数为：

$$A = a^2 + 2vbc, \quad B = 2ab + vc^2, \quad C = b^2 + 2ac.$$

直接这样计算就要做三次乘方和三次乘法，不够划算，所以要想办法消掉这里的交叉项 ab, ac, bc 。当然这里还没有利用 $g \in G_{\Phi_6}(\mathbb{F}_q)$ 这个特殊性质。注意到该条件等价于：

$$g \in G_{\Phi_6}(\mathbb{F}_q) \iff g^{q^2-q+1} = 1 \iff g^{q^2}g = g^q$$

我们代入 $g = a + bw + cw^2$ ，展开并计算 $g^{q^2}g$ 和 g^q 的常数项相同是否能推出代数关系。

我们设 $\omega = w^{q-1} = \frac{w^q}{w}$, 则 $\omega^3 = (\frac{w^q}{w})^3 = \frac{v^q}{v} = -1$, 因此 $\omega^6 = 1$ 。

如果 $\omega = -1$, 否则 $w^q = -w$ 会推出 $w^{q^2} = (-w)^q = -w^q = w$, 从而 $w \in \mathbb{F}_{q^2}$, 与 w 用来生成三次扩张 $\mathbb{F}_{q^6}/\mathbb{F}_{q^2}$ 矛盾!

而之前已经算出 $\omega^3 = -1$, 6 的非平凡因子只有 2 和 3, w 不是本原的 2 次或 3 次单位根, 因此只能是 6 次。

然后我们证明 $\omega \in \mathbb{F}_q$, 由于 $q = p^2 \equiv 1 \pmod{6}$, 所以 $6 \mid q - 1$ 。而 $\mathbb{F}_q^*$ 是阶为 $q - 1$ 的循环群, 因此一定包含六次单位根 ω 。

回到计算 g^q, g^{q^2} 的常数项本身, 由于 $\omega \in \mathbb{F}_q$, 可得 $\omega^q = \omega$ 。而前文定义 $w^q = \omega w$, 故 $w^{q^2} = \omega^q w^q = \omega^2 w$ 。则:

$$g^q = (a + bw + cw^2)^q = \bar{a} + \bar{b}\omega w + \bar{c}\omega^2 w^2$$

$$g^{q^2} = (a + bw + cw^2)^{q^2} = a + b\omega^2 w + c\omega^4 w^2$$

计算 $g^{q^2}g = (a + b\omega^2 w + c\omega^4 w^2)(a + bw + cw^2)$ 的常数项 (注意变量是 w , 利用 $w^3 = v$ 这个关系):

$$a \times a = a^2, \quad (b\omega^2 w)(cw^2) = vbc\omega^2, \quad (\omega^4 w^2)(bw) = vbc\omega^4$$

由于 ω 是本原六次单位根, $\omega^4 + \omega^2 + 1 = 0$, 故:

$$a^2 + vbc\omega^2 + vbc\omega^4 = a^2 + vbc(\omega^4 + \omega^2) = a^2 - vbc$$

也就是 $g^{q^2}g$ 的常数项是 $a^2 - vbc$, 而上一页已经推出 g^q 的常数项是 $\bar{a}$, 故我们推出了第一个恒等式:

$$vbc = a^2 - \bar{a}$$

刚才那个恒等式只是常数项相等推出来的关系, 对于 w 和 w^2 的系数相等的关系, 我们也可以推出剩下两个:

$$ab = vc^2 + \bar{b}, \quad ac = b^2 - \bar{c}$$

我们带回之前的平方公式结果 $A + Bw + Cw^2$ 对应的系数:

$$A = a^2 + 2vbc, \quad B = 2ab + vc^2, \quad C = b^2 + 2ac.$$

就可以消去交叉项, 得到最终的 Granger-Scott 公式:

$$A = 3a^2 - 2\bar{a}, \quad B = 3vc^2 + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}.$$

Cost

$$f \in \mathbb{F}_{p^{12}}^*$$

$$f^{(p^{12}-1)/r} = \left(f^{(p^6-1)(p^2+1)} \right)^{(p^4-p^2+1)/r}$$

easy part

$$g = f^{(p^6-1)(p^2+1)}$$

$$g \in G_{\Phi_6}(\mathbb{F}_{p^2}), \quad g^{p^4-p^2+1} = 1$$

hard part: 反复平方

$$g = a + bw + cw^2, \quad a, b, c \in \mathbb{F}_{p^4}$$

$$g^2 = A + Bw + Cw^2$$

$$A = 3a^2 - 2\bar{a}$$

$$B = 3c^2v + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}$$

Granger-Scott

$$\text{主要成本} = a^2 + b^2 + c^2 = 3S_4 = 6S_2 + 3M_2$$



three $\mathbb{F}_{p^4}$ squarings

$$(B, C) : \quad (2 \times 2 \times 2 \times 1)\text{-way}$$

$$(2 \times 1 \times 4 \times 1)\text{-way}$$

$$A : \quad (1 \times 2 \times 2 \times 2)\text{-way}$$

$$(1 \times 1 \times 4 \times 2)\text{-way}$$



hybrid vectorization

$$x \in \mathbb{F}_p : \quad x = \sum_{i=0}^7 x_i 2^{48i}$$

$$\text{vpmadd52luq/huq}$$



IFMA / 48-bit limb

上一节我们通过推导 Granger-Scott 公式，把 cyclotomic squaring 中的三次平方和三次乘法操作，减少到仅有的三次 $\mathbb{F}_{q^2} = \mathbb{F}_{p^4}$ 的平方操作，也就是给出 $a, b, c \in \mathbb{F}_{p^4}$ ，求 a^2, b^2, c^2 。

为了方便，我们记在 $\mathbb{F}_{p^k}$ 的平方操作代价 S_k ，乘法操作代价为 M_k ，所以一次 cyclotomic squaring 的代价就是 $3S_4$ 。而前文我们推导出在这个扩域 $\mathbb{F}_{q^2} = \mathbb{F}_q[v]/(v^2 - \xi)$ 中：

$$x^2 = (a + bv)^2 = (a^2 + \xi b^2) + 2abv$$

也就是一次 $S_4 = 2S_2 + M_2$ ，那么整个成本就是 $3S_4 = 6S_2 + 3M_2$ 。假如我们要计算 x_1^2, x_2^2, x_3^2 ，我们可以把这九个“真正的计算”列成三行：

$$a_1^2, \quad b_1^2, \quad a_1b_1$$

$$a_2^2, \quad b_2^2, \quad a_2b_2$$

$$a_3^2, \quad b_3^2, \quad a_3b_3$$

接下来面临的就是一个工程问题了，也马上进入本文真正的 contribution：

这三个 $\mathbb{F}_{p^4}$ squaring 数量是奇数，怎样把它们放进 AVX-512 的并行布局里，尽量不浪费 lanes？

Hybrid vectorization

$$f \in \mathbb{F}_{p^{12}}^*$$

$$f^{(p^{12}-1)/r} = \left(f^{(p^6-1)(p^2+1)} \right)^{(p^4-p^2+1)/r}$$

easy part

$$g = f^{(p^6-1)(p^2+1)}$$

$$g \in G_{\Phi_6}(\mathbb{F}_{p^2}), \quad g^{p^4-p^2+1} = 1$$

hard part: 反复平方

$$g = a + bw + cw^2, \quad a, b, c \in \mathbb{F}_{p^4}$$

$$g^2 = A + Bw + Cw^2$$

$$A = 3a^2 - 2\bar{a}$$

$$B = 3c^2v + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}$$

Granger-Scott

$$\text{主要成本} = a^2 + b^2 + c^2 = 3S_4 = 6S_2 + 3M_2$$

three $\mathbb{F}_{p^4}$ squarings

$$(B, C) : \quad \begin{array}{l} (2 \times 2 \times 2 \times 1)\text{-way} \\ (2 \times 1 \times 4 \times 1)\text{-way} \end{array}$$

$$A : \quad \begin{array}{l} (1 \times 2 \times 2 \times 2)\text{-way} \\ (1 \times 1 \times 4 \times 2)\text{-way} \end{array}$$

hybrid vectorization

$$x \in \mathbb{F}_p : \quad x = \sum_{i=0}^7 x_i 2^{48i}$$

$$\text{vpmadd52luq/huq}$$

IFMA / 48-bit limb

我们定义 $(w \times x \times y \times z) - \text{way}$ 表示在一个 $\mathbb{F}_{p^4}$ 层的操作：

- 同时计算 w 个 $\mathbb{F}_{p^4}$ 操作；
- 每一个 $\mathbb{F}_{p^4}$ 操作内，同时计算 x 个 $\mathbb{F}_{p^2}$ 操作；
- 每一个 $\mathbb{F}_{p^2}$ 操作内，同时计算 y 个 $\mathbb{F}_p$ 操作；
- 每一个 $\mathbb{F}_p$ 操作内，同时使用 z 个 64-bit SIMD lanes；
- 同时保证 $wxyz = 8$ ，因为一个 ZMM 寄存器有 8 个 64-bit SIMD lanes.

然而我们看到，前面的乘法操作是 $3S_4$ ，并不是 1/2/4/8 中的任意一个数，不能完整地填满整个 ZMM 寄存器。因此作者提出了一种名为 hybrid vectorization 的新技术，简而言之就是在**某一个计算 level**，将某些结构相似的算式绑定在一起向量化，而让剩下的那一个做 1-way 布局，并将 SIMD 并行性继续下沉到更低层。

首先拿 Granger-Scott 这一层（也就是最上面那一层）为例。我们有三种路径 A, B, C ：

$$A = 3a^2 - 2\bar{a}, \quad B = 3vc^2 + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}.$$

式子 A 只涉及到了 a 自身，而 B 和 C 有一种对 b, c 的轮换感。所以 Cheng 选择将 B, C 这两个合并在一起进行 SIMD 运算。

现在我们考虑 (B, C) 联合计算的情况，也就是最高层的 Granger-Scott squaring，对应公式：

$$B = 3c^2v + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}$$

可以抽象为以下伪代码（对应了 (w, x, y, z) 四元组的 $w = 2$ ）：

```
Function: CYC_SQR_BC_2WAY(b, c)
Input: b, c ∈ F_{p^4}

(tb, tc) ← SQR_FP4_2WAY(b, c)
(B, C) ← 3·(v·tc, tb) + 2·(conj(b), -conj(c))

Output: (B, C)
```

对于另一个 A ，就直接采用 1-way $\mathbb{F}_{p^4}$ squaring，对应 A 这边的 $w = 1$ ：

```
Function: CYC_SQR_A_1WAY(a)
Input: a ∈ F_{p^4}

ta ← SQR_FP4_1WAY(a)
A ← 3·ta - 2·conj(a)

Output: A
```

然后考虑下面一层，也就是 $\mathbb{F}_{p^2} \rightarrow \mathbb{F}_{p^4}$ ，即 $\mathbb{F}_{p^4}$ squaring，扩域为 $\mathbb{F}_{q^2} = \mathbb{F}_q[v]/(v^2 - \xi)$ 。

在第四节我们讲过， $x^2 = (a^2 + \xi b^2) + 2abv$ ，真正的成本就是这三部分的计算：

$$a_0^2, \quad b_0^2, \quad a_0 b_0$$

思路与上面那个 cyclotomic squaring 一致，我们显然要将 a_0^2, b_0^2 同时做 SIMD，而交叉乘法 $a_0 b_0$ 则单独处理。上一层 square (B, C) 部分的伪代码如下：

```
Function: SQR_FP4_2WAY(b, c)
Input:
    b = b0 + b1·v ∈ F_{p^4}
    c = c0 + c1·v ∈ F_{p^4}

(sb0, sb1, sc0, sc1)
    ← SQR_FP2_4WAY(b0, b1, c0, c1)

(mb, mc)
    ← MUL_FP2_2WAY((b0, b1), (c0, c1))

tb ← (sb0 + ξ·sb1) + (2·mb)·v
tc ← (sc0 + ξ·sc1) + (2·mc)·v

Output: (tb, tc)
```


对于上一层的 1-way 分支，也就是单独处理 A 路径中 a^2 部分，对应以下代码：

```
Function: SQR_FP4_1WAY(a)
Input: a = a0 + a1·v ∈ F_{p^4}

(s0, s1) ← SQR_FP2_2WAY(a0, a1)
m        ← MUL_FP2_1WAY(a0, a1)

ta ← (s0 + ξ·s1) + (2·m)·v

Output: ta
```

现在我们就来到了 $\mathbb{F}_p \rightarrow \mathbb{F}_{p^2}$ 这一层，相当于分裂成了四个原语：

- 前者 `SQR_FP4_2WAY` 需要进一步实现 `SQR_FP2_4WAY` 和 `MUL_FP2_2WAY`；
- 后者 `SQR_FP4_1WAY` 则需要实现 `SQR_FP2_2WAY` 和 `MUL_FP2_1WAY`。

这里作者为了简化，并没有真的去逐一写出四个原语的实现，而是统一为实现 `SQR_FP2_*WAY` 和 `MUL_FP2_*WAY`。

区别只是同时喂进去多少组输入，以及每个 $\mathbb{F}_p$ 运算占几个 lanes。

首先考虑简单一点的 SQR_FP2_*WAY。数学方面，设 $\alpha_0, \alpha_1 \in \mathbb{F}_p$, $u^2 = -1$ ，则 $\mathbb{F}_{p^2}$ 的元素则可表示为 $\alpha_0 + \alpha_1 u$ ，平方展开为：

$$(\alpha_0 + \alpha_1 u)^2 = (\alpha_0 + \alpha_1)(\alpha_0 - \alpha_1) + (2\alpha_0 \alpha_1)u$$

也就是我们令输出 $r_0 = \alpha_0^2 - \alpha_1^2$, $r_1 = 2\alpha_0 \alpha_1$ 即可。

SIMD 实现方面，假如同时做 $j = 4$ 个元素的乘法，那么输入为 $\alpha_0^{(j)}, \alpha_1^{(j)}$ ，我们也可以得到输出就是 $r_0^{(j)} = (\alpha_0^{(j)} + \alpha_1^{(j)})(\alpha_0^{(j)} - \alpha_1^{(j)})$, $r_1^{(j)} = 2\alpha_0^{(j)} \alpha_1^{(j)}$ ，并可立即写出 SIMD 伪代码：

```
Function: SQR_FP2_4WAY( $\alpha^{(1)}, \dots, \alpha^{(4)}$ )
Input:  $\alpha^{(j)} = \alpha_0^{(j)} + \alpha_1^{(j)} \cdot u \in \mathbb{F}_{p^2}$ 

( $t_0^{(1)}, \dots, t_0^{(4)}$ )  $\leftarrow$  ( $\alpha_0^{(1)} + \alpha_1^{(1)}$ , ...,  $\alpha_0^{(4)} + \alpha_1^{(4)}$ )
( $t_1^{(1)}, \dots, t_1^{(4)}$ )  $\leftarrow$  ( $\alpha_0^{(1)} - \alpha_1^{(1)}$ , ...,  $\alpha_0^{(4)} - \alpha_1^{(4)}$ )

( $r_0^{(1)}, r_1^{(1)}, \dots, r_0^{(4)}, r_1^{(4)}$ )
 $\leftarrow$  MUL_FP_8WAY(( $t_0^{(1)}, 2\alpha_0^{(1)}, \dots, t_0^{(4)}, 2\alpha_0^{(4)}$ ),
                  ( $t_1^{(1)}, \alpha_1^{(1)}, \dots, t_1^{(4)}, \alpha_1^{(4)}$ ))

Output:  $r^{(j)} = r_0^{(j)} + r_1^{(j)} \cdot u, \quad j = 1, \dots, 4$ 
```

然后考虑 MUL_FP2_*WAY。这里有两个元素 $\alpha = \alpha_0 + \alpha_1 u, \beta = \beta_0 + \beta_1 u$ ，普通展开可得：

$$\alpha\beta = (\alpha_0\beta_0 - \alpha_1\beta_1) + (\alpha_0\beta_1 + \alpha_1\beta_0)u$$

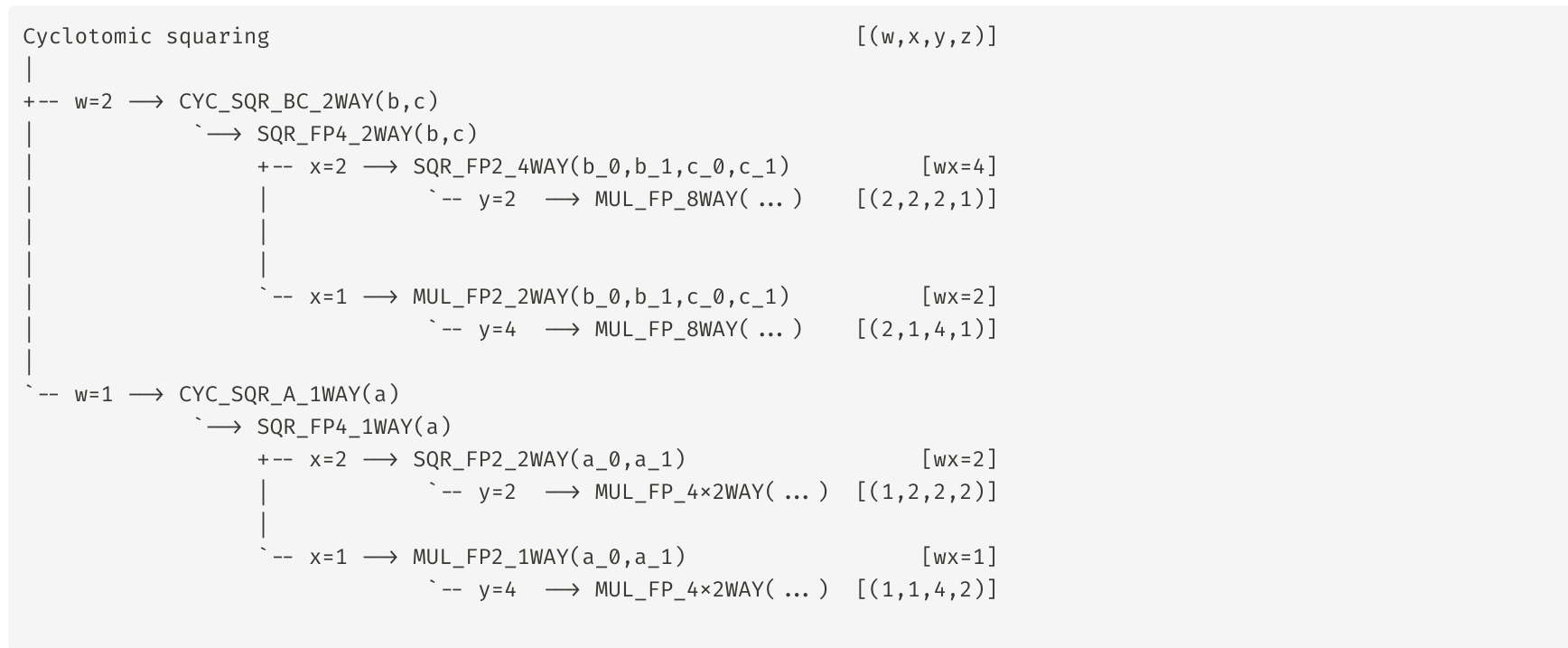
暴露了四个独立的 multiplication。将其向量化就是 $p_0^{(j)} = \alpha_0^{(j)} \beta_0^{(j)}, p_1^{(j)} = \alpha_1^{(j)} \beta_1^{(j)}, p_2^{(j)} = \alpha_0^{(j)} \beta_1^{(j)}, p_3^{(j)} = \alpha_1^{(j)} \beta_0^{(j)}$ 。当 $j = 2$ 时，代码如下：

```
Function: MUL_FP2_2WAY( $\alpha^{(1)}$ ,  $\beta^{(1)}$ ,  $\alpha^{(2)}$ ,  $\beta^{(2)}$ )
Input:
     $\alpha^{(j)} = \alpha_0^{(j)} + \alpha_1^{(j)} \cdot u \in F_{\{p^2\}}$ 
     $\beta^{(j)} = \beta_0^{(j)} + \beta_1^{(j)} \cdot u \in F_{\{p^2\}}$ 

( $p_0^{(1)}$ ,  $p_1^{(1)}$ ,  $p_2^{(1)}$ ,  $p_3^{(1)}$ ,
  $p_0^{(2)}$ ,  $p_1^{(2)}$ ,  $p_2^{(2)}$ ,  $p_3^{(2)}$ )
     $\leftarrow$  MUL_FP_8WAY(           // 竖着看
        ( $\alpha_0^{(1)}$ ,  $\alpha_1^{(1)}$ ,  $\alpha_0^{(1)}$ ,  $\alpha_1^{(1)}$ ,  $\alpha_0^{(2)}$ ,  $\alpha_1^{(2)}$ ,  $\alpha_0^{(2)}$ ,  $\alpha_1^{(2)}$ ),
        ( $\beta_0^{(1)}$ ,  $\beta_1^{(1)}$ ,  $\beta_1^{(1)}$ ,  $\beta_0^{(1)}$ ,  $\beta_0^{(2)}$ ,  $\beta_1^{(2)}$ ,  $\beta_1^{(2)}$ ,  $\beta_0^{(2)}$ )
    )

Output:
     $r^{(j)} = (p_0^{(j)} - p_1^{(j)})$ 
              $+ (p_2^{(j)} + p_3^{(j)}) \cdot u, \quad j = 1, 2$ 
```

总结一下，我们可以把 cyclotomic squaring 操作的几个函数的调用关系图画为：



这个问题最终被抽象成了 `MUL_FP_8WAY` 和 `MUL_FP_4x2WAY` 在具体 SIMD 上的实现，也就是下一节的内容。

IFMA / 48-bit limb

$$f \in \mathbb{F}_{p^{12}}^*$$

$$f^{(p^{12}-1)/r} = \left(f^{(p^6-1)(p^2+1)} \right)^{(p^4-p^2+1)/r}$$

easy part

$$g = f^{(p^6-1)(p^2+1)}$$

$$g \in G_{\Phi_6}(\mathbb{F}_{p^2}), \quad g^{p^4-p^2+1} = 1$$

hard part: 反复平方

$$g = a + bw + cw^2, \quad a, b, c \in \mathbb{F}_{p^4}$$

$$g^2 = A + Bw + Cw^2$$

$$A = 3a^2 - 2\bar{a}$$

$$B = 3c^2v + 2\bar{b}, \quad C = 3b^2 - 2\bar{c}$$

Granger-Scott

$$\text{主要成本} = a^2 + b^2 + c^2 = 3S_4 = 6S_2 + 3M_2$$

three $\mathbb{F}_{p^4}$ squarings

$$(B, C) : \quad \begin{array}{l} (2 \times 2 \times 2 \times 1)\text{-way} \\ (2 \times 1 \times 4 \times 1)\text{-way} \end{array}$$

$$A : \quad \begin{array}{l} (1 \times 2 \times 2 \times 2)\text{-way} \\ (1 \times 1 \times 4 \times 2)\text{-way} \end{array}$$

hybrid vectorization

$$x \in \mathbb{F}_p : \quad x = \sum_{i=0}^7 x_i 2^{48i}$$

vpmadd521uq/huq

IFMA / 48-bit limb

在正式讲这两个函数之前，我们先了解一下，SIMD 在具体的硬件中是如何进行加法和乘法操作的。假如我们要同时操作 8 个 384-bit 的数 $a^{(j)}$, $j = 1 \cdots 8$ 。注意这里的 $a^{(j)}$ 不是整体存放在一个 ZMM 寄存器中，而是把每个 $a^{(j)}$ 的 48 位切片，从低到高依次存放到 ZMM A_0 到 ZMM A_7 中，如图所示：

	8 个并行任务 / SIMD lanes				
	(1)	(2)	(3)	...	(8)
ZMM A_0, limb 0	$a_0^{(1)}$	$a_0^{(2)}$	$a_0^{(3)}$	...	$a_0^{(8)}$
ZMM A_1, limb 1	$a_1^{(1)}$	$a_1^{(2)}$	$a_1^{(3)}$	...	$a_1^{(8)}$
ZMM A_2, limb 2	$a_2^{(1)}$	$a_2^{(2)}$	$a_2^{(3)}$	...	$a_2^{(8)}$
...					
ZMM A_7, limb 7	$a_7^{(1)}$	$a_7^{(2)}$	$a_7^{(3)}$	...	$a_7^{(8)}$

如果我们要同时计算 $a^{(j)}$ 与 $b^{(j)}$ 这 $8 + 8 = 16$ 个数的加法（假设 $b^{(j)}$ 被分别存放在 ZMM B_0 到 ZMM B_7 中），SIMD 会使用 $C_0 \leftarrow \text{VPADDQ}(A_0, B_0)$ 计算出低 48 位的结果 C_0 ，进位会被丢弃。

然后执行以下步骤之后，就完成了同时对 $a^{(j)}$ 和 $b^{(j)}$ 这 $8 + 8 = 16$ 个数的加法操作：

$C_0 \leftarrow \text{VPADDQ}(A_0, B_0)$	$C_0 \leftarrow A_0 + B_0$
$C_1 \leftarrow \text{VPADDQ}(A_1, B_1)$	$C_1 \leftarrow A_1 + B_1$
...	
$C_7 \leftarrow \text{VPADDQ}(A_7, B_7)$	$C_7 \leftarrow A_7 + B_7$

那可能有同学会问：那为什么要搞这么麻烦，每个元素存一个 ZMM 寄存器，ZMM 直接相加不也能实现同样的 8 次 `VPADDQ` 操作吗？

答案也确实如此。但如果是要做乘法操作，就不是那么简单了。我们先介绍“官方的 SIMD 乘法操作”：

由于乘法本质上也可以写成卷积的形式，需要将两个乘数的每一个 limb 执行乘法指令，一共要执行 $8 \times 8 = 64$ 次。举一个例子，假如选择 limb $i = 0, j = 3$ ，也就是选出了这两行：

$$A_0 = | a_0^{(1)} | a_0^{(2)} | \dots | a_0^{(8)} | \qquad B_3 = | b_3^{(1)} | b_3^{(2)} | \dots | b_3^{(8)} |$$

那么执行一条向量乘法：

$$P_3 \leftarrow \text{IFMA}(A_0, B_3)$$

会得到：

$$P_3 = | a_0^{(1)}b_3^{(1)} | a_0^{(2)}b_3^{(2)} | \dots | a_0^{(8)}b_3^{(8)} |$$

当然 48-bit 的乘法结果肯定会超出 limb 的 64-bit 范围，因次实际上这个指令由两个 intrinsic 完成：

`vpadd52luq` `vpadd52huq`，分别对应做 52-bit 的乘法，保留结果的低 52 位和高 52 位。

对于乘法局部结果 P_k ，我们需要通过 IFMA 指令来做卷积：

$$P_k = \sum_{i+j=k} \text{IFMA}(A_i, B_j)$$

其中 $0 \leq k \leq 14$ ，对应 limb 0-14，进位的话分布在 limb 1-15，所以处理进位之后，把这 16 个 limb 从低到高拼起来即可同时得到 8 个元素的乘法结果。注意到在 AVX512-IFMA 指令集中，程序员可见的 ZMM 寄存器有 32 个，而 $8 + 8 + 16 = 32$ ，所以做 8 个 $\mathbb{F}_p$ 乘法刚好够用。

回到之前的问题：现在如果还使用“每个元素存一个 ZMM 寄存器”的存放方案，还可以方便实现乘法吗？
这个问题留作课后思考。

然后我们回到这个式子，如果老老实实做 64 次 IFMA 乘法，效率肯定不高。

于是 Cheng 选择把 8-limb 的乘法用 Karatsuba 分成 4-limb 的三次乘法。也就是卷积做：

$$\sum \text{IFMA}(A_L, B_L) \quad \sum \text{IFMA}(A_H, B_H) \quad \sum \text{IFMA}(A_L + A_H, B_L + B_H)$$

首先我们考虑 4-limb 卷积的代码（见下页）：


```
Function: CONV4_IFMA(A_0, ... , A_3, B_0, ... , B_3)
```

```
Input: A_i, B_i ∈ ZMM
```

```
for k = 0, ... , 6:
```

```
    L_k ← 0
```

```
    H_k ← 0
```

```
    for all i+j=k:
```

```
        L_k ← VMACLO(L_k, A_i, B_j)
```

```
        H_k ← VMACHI(H_k, A_i, B_j)
```

```
Z_0 ← L_0
```

```
for k = 1, ... , 6:
```

```
    Z_k ← L_k + (H_{k-1} << 4)
```

```
Z_7 ← H_6 << 4
```

```
Output: (Z_0, ... , Z_7)
```

注意这里左移四位是因为 VFMA 指令会截断掉后面的 52-bit，但我们实际上是 48-bit 一个 limb，这样实际上只留了高 $96 - 52 = 44$ -bit，需要左移四位来恢复正确的 limb 数值。

但左移四位仍然有一个问题，就是先前右移被截断的 $[48, 52)$ 位并不会立即恢复。作者为了流水式拼接，选择不在现在加回这四位，而是在最后的 carry propagation 阶段一并处理。

有了四位卷积的内核，我们就可以实现 `MUL_FP_8WAY` 和 `MUL_FP_4x2WAY` 了，我们先看前者。我们假设 8-lane 输入 $\alpha^{(j)}, \beta^{(j)}$ 被分成了两半 $\alpha^{(j)} = \alpha_L^{(j)} + R\alpha_H^{(j)}$, $R = 2^{48}$ ，则直接套用 Karatsuba 公式：

```
Function: MUL_FP_8WAY( $\alpha^{(1)}, \dots, \alpha^{(8)}, \beta^{(1)}, \dots, \beta^{(8)}$ )
```

```
Input:
```

```
  A_i =  $\langle \alpha_i^{(1)}, \dots, \alpha_i^{(8)} \rangle$ , i = 0, ..., 7
```

```
  B_i =  $\langle \beta_i^{(1)}, \dots, \beta_i^{(8)} \rangle$ , i = 0, ..., 7
```

```
Z_L  $\leftarrow$  CONV4_IFMA(A_0, ..., A_3, B_0, ..., B_3)
```

```
Z_H  $\leftarrow$  CONV4_IFMA(A_4, ..., A_7, B_4, ..., B_7)
```

```
for i = 0, ..., 3:
```

```
  A'_i  $\leftarrow$  A_i + A_{i+4}
```

```
  B'_i  $\leftarrow$  B_i + B_{i+4}
```

```
Z_M  $\leftarrow$  CONV4_IFMA(A'_0, ..., A'_3, B'_0, ..., B'_3)
```

```
Z_M  $\leftarrow$  Z_M - Z_L - Z_H
```

```
T  $\leftarrow$  Z_L +  $R^4 \cdot Z_M$  +  $R^8 \cdot Z_H$ 
```

```
Output:
```

```
  tt^{(j)} =  $\alpha^{(j)}\beta^{(j)}$ , j = 1, ..., 8
```

```
  // double-length, unreduced
```

MUL_FP_4x2WAY 相对复杂一点，这里的 α_i 不再是按照 limb 存成一行 1×8 的形式，而是类似于 2×4 的存储方式：

lanes:		0	1		2	3		4	5		6	7	
		task 1			task 2			task 3			task 4		
		$\alpha_{\{0\}} \alpha_{\{4\}}$			...								
		$\alpha_{\{1\}} \alpha_{\{5\}}$			...								
		$\alpha_{\{2\}} \alpha_{\{6\}}$			...								
		$\alpha_{\{3\}} \alpha_{\{7\}}$			...								

然后介绍两个硬件重排函数 SHUF_LO, SHUF_HI，前者的作用是把每一对元素中左边那个复制两次，后者就是右边那个复制两次，形如对原来位于 ZMM 寄存器的向量 X ：

lane:		0	1	2	3	4	5	6	7
X	=	A	B	C	D	E	F	G	H

做 SHUF_LO 会得到 AACCEEGG，而做 SHUF_HI 会得到 BBDDFFHH。

然后我们看对应代码（见下页）：

Function: MUL_FP_4x2WAY($\alpha^{(1)}, \dots, \alpha^{(4)},$
 $\beta^{(1)}, \dots, \beta^{(4)}$)

Input:

for $i = 0, \dots, 3$

$A_i = \langle \alpha_{\{2i\}}^{(1)}, \alpha_{\{2i+1\}}^{(1)}, \dots, \alpha_{\{2i\}}^{(4)}, \alpha_{\{2i+1\}}^{(4)} \rangle$

$B_i = \langle \beta_{\{2i\}}^{(1)}, \beta_{\{2i+1\}}^{(1)}, \dots, \beta_{\{2i\}}^{(4)}, \beta_{\{2i+1\}}^{(4)} \rangle$

$L_0, \dots, L_{10} \leftarrow 0; H_0, \dots, H_{10} \leftarrow 0$

for $j = 0, \dots, 3$:

$B_{\text{even}} \leftarrow \text{SHUF_LO}(B_j); \quad B_{\text{odd}} \leftarrow \text{SHUF_HI}(B_j)$

for $i = 0, \dots, 3$:

$L_{\{i+j\}} \leftarrow \text{VMACLO}(L_{\{i+j\}}, A_i, B_{\text{even}});$

$L_{\{i+j+4\}} \leftarrow \text{VMACLO}(L_{\{i+j+4\}}, A_i, B_{\text{odd}})$

$H_{\{i+j\}} \leftarrow \text{VMACHI}(H_{\{i+j\}}, A_i, B_{\text{even}});$

$H_{\{i+j+4\}} \leftarrow \text{VMACHI}(H_{\{i+j+4\}}, A_i, B_{\text{odd}})$

$Z_0 \leftarrow L_0$

for $k = 1, \dots, 10$:

$Z_k \leftarrow L_k + (H_{\{k-1\}} \ll 4)$

$Z_{11} \leftarrow H_{10} \ll 4$

Output:

$tt^{(j)} = \alpha^{(j)}\beta^{(j)}, j = 1, \dots, 4$

// paired-lane double-length representation

中间循环的过程可能有点抽象，这里展开一下。对于四个独立乘法任务的 A_i 和 B_j 分块，令：

$$\begin{aligned} A_i &= \begin{bmatrix} a & b & c & d & e & f & g & h \end{bmatrix} = \begin{bmatrix} \alpha_i^{(1)} & \alpha_{i+4}^{(1)} & \dots \end{bmatrix} \\ B_j &= \begin{bmatrix} A & B & C & D & E & F & G & H \end{bmatrix} = \begin{bmatrix} \beta_i^{(1)} & \beta_{i+4}^{(1)} & \dots \end{bmatrix} \end{aligned}$$

假如我们拆分乘数 B_j ，shuffle 之后：

$$\begin{aligned} B_{\text{even}} &= \text{SHUF_LO}(B_j) & B_{\text{odd}} &= \text{SHUF_HI}(B_j) \\ &= \begin{bmatrix} A & A & C & C & E & E & G & G \end{bmatrix} & & = \begin{bmatrix} B & B & D & D & F & F & H & H \end{bmatrix} \end{aligned}$$

然后分别和 A_i 逐 lane 相乘：

$$\begin{aligned} A_i \times B_{\text{even}} & & A_i \times B_{\text{odd}} \\ = \begin{bmatrix} aA & bA & cC & dC & eE & fE & gG & hG \end{bmatrix} & & = \begin{bmatrix} aB & bB & cD & dD & eF & fF & gH & hH \end{bmatrix} \end{aligned}$$

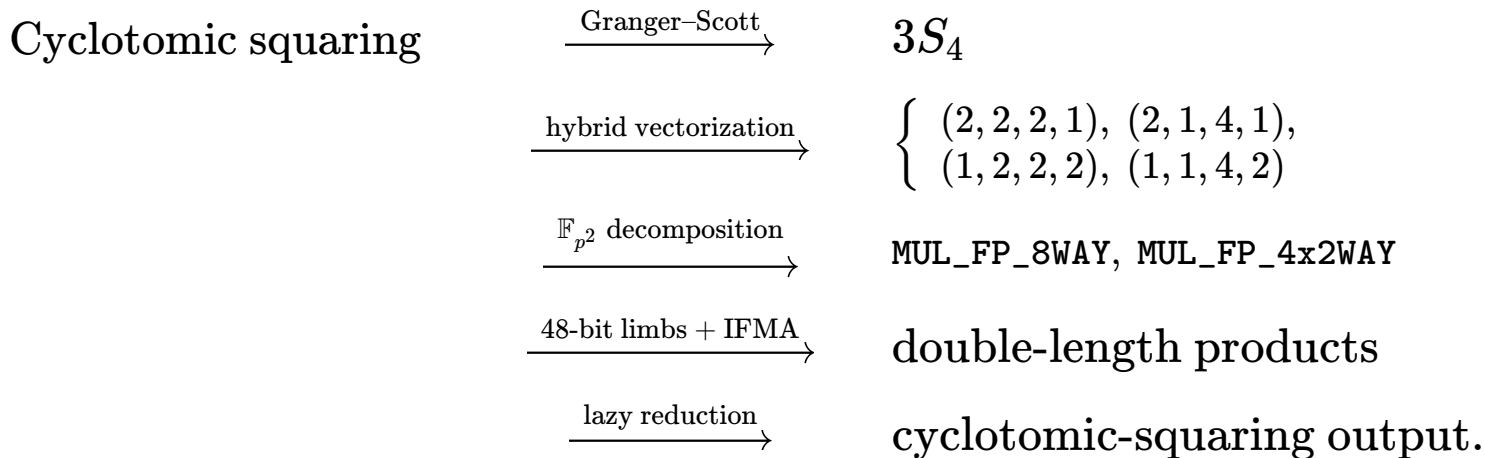
然后考虑数位问题，对于 $A_i \times B_{\text{even}}$ ，对应的数位是 i, j ，因此结果该进入 L_{i+j} 和 H_{i+j} ；对于 $A_i \times B_{\text{odd}}$ 对应的数位是 $i, j + 4$ ，因此结果该进入 L_{i+j+4} 和 H_{i+j+4} 。最后再做 carry-propagation 即可。

注意这里没有用到 Karatsuba，而且有大量 shuffle 操作，因此效率不如 `MUL_FP_8WAY`。

一些细碎的优化（从底层到高层说）

- 每个基域元素采用 8 个 48-bit limbs，而不是将 IFMA 支持的 52 bits 完全用满，从而留下 4 bits headroom。这样，Karatsuba 和扩域公式中的一些 limb-wise 加法之后，操作数仍可直接送入 IFMA。
- 在基域乘法中，先用 IFMA 按 limb pair 扫描卷积，并将 partial products 直接累加到对应的 double-length accumulator；最后再统一进行 carry propagation。IFMA 自带 fused accumulation，因此不需要为每个 partial product 额外执行向量加法，有利于减少 port congestion 和调度压力。
- 在 $\mathbb{F}_{p^4}$ squaring 中，作者采用 $S_4 = 2S_2 + M_2$ 的 schoolbook 公式，而不是成本为 $3S_2$ 的 Karatsuba 公式，因为前者在向量实现中需要更少的加减法。
- 底层 $\mathbb{F}_{p^2}$ multiplication 和 squaring 可以暂时保留 double-length 中间结果，不立即做模约减；等到一次 $\mathbb{F}_{p^4}$ squaring 的末尾再统一 reduction。这会增加 blend 和 permutation 开销，但实测 lazy reduction 仍然更快。
- Cyclotomic squaring 的输出 A 与 (B, C) 保持和输入 a 与 (b, c) 相同的向量布局。因此连续执行 squaring 时，上一轮输出可以直接作为下一轮输入，不需要额外的 vector transformation。
- Granger–Scott full squaring 中， A 分支的并行程度低于 (B, C) 分支。作者因此也集成了已有的 compressed cyclotomic squaring：连续平方时仅保存压缩表示 $\mathcal{C}(g) = (b, c)$ ，并只更新 B, C ，暂时省去 A 。当需要把选定的 g^{2^i} 相乘以构造 $g^{|z|}$ 时，再通过一次 batched decompression 恢复缺失的 A 分量。

From algebra to AVX-512IFMA



至此，Granger-Scott 给出的三个 $\mathbb{F}_{p^4}$ 平方，已经被逐层分解为两种具体的 AVX-512IFMA 基域乘法内核。Hybrid vectorization 的核心不是提出新的域运算公式，而是在不同扩域层之间重新分配并行度，使八个 SIMD lanes 始终尽可能被有效利用。接下来通过实验观察，这种布局是否真的减少了 transformation 和 reduction 开销，并最终改善完整 pairing 的性能。

六个 primitive 的优化选择

Primitive	公式 / 方法	向量化思路
cyclotomic_sqr_fp12	Granger-Scott	Hybrid: 单独计算 A ; 把 B, C 合并做 2-way $\mathbb{F}_{p^4}$ 平方。
mul_fp12	Karatsuba	Hybrid: 两个 $\mathbb{F}_{p^6}$ 乘法并行; 第三个乘法单独做 1-way。
mul_by_xy00z0_fp12	稀疏 schoolbook	在 $\mathbb{F}_{p^2}$ 层做 4-way; 两个稀疏 $\mathbb{F}_{p^6}$ 乘法并行。
sqr_fp12	Karatsuba	在 $\mathbb{F}_{p^2}$ 层做 4-way; 复用 2-way $\mathbb{F}_{p^6}$ 乘法 kernel。
line_dbl	dbl-2009-l	在 $\mathbb{F}_{p^2}$ 层做 4-way; 提高 point doubling 与 line computation 的 lane 利用率。
line_add	madd-2007-bl	在 $\mathbb{F}_{p^2}$ 层做 2-way; 用于 sparse $\mathbb{F}_{p^{12}}$ multiplication 之前。

Evaluation: high-level primitives

Intel Core i3-1005G1 (Ice Lake), GCC 13.3, `-O3`, TurboBoost disabled · baseline: scalar x64 assembly in blst

Operation	blst cycles	avxbbs cycles	Speed-up
$\mathbb{F}_{p^{12}}$ cyclotomic squaring	3,313	1,789	1.85×
$\mathbb{F}_{p^{12}}$ compressed cyclotomic squaring	2,346	1,088	2.16×
$\mathbb{F}_{p^{12}}$ multiplication	7,653	4,074	1.88×
$\mathbb{F}_{p^{12}}$ sparse multiplication	5,057	2,373	2.13×
$\mathbb{F}_{p^{12}}$ squaring	5,456	2,412	2.26×
<code>line_dbl</code>	4,127	2,141	1.93×
<code>line_add</code>	5,293	3,125	1.69×

Evaluation: end-to-end pairing

Cycles on Intel Core i3-1005G1 · speed-up relative to the original scalar blst Granger–Scott baseline

Computation	blst	avxbls	Speed-up
Miller loop	1,003,400	499,393	2.01×
Final exponentiation (Granger–Scott)	1,349,003	762,186	1.77×
Final exponentiation (compressed)	1,169,728	694,729	1.94×
Full pairing (Granger–Scott)	2,351,615	1,265,314	1.86×
Full pairing (compressed)	2,169,338	1,195,236	1.97×

1.86×

Granger–Scott full pairing

1.97×

compressed full pairing

Discussion

有效之处

- 微基准中的加速能够传递到完整 pairing。
- Miller loop 达到 2.01× 加速，接近各个优化 primitive 的加速区间。
- Compressed cyclotomic squaring 省去了系数 A ，使其加速比从 1.85× 提升到 2.16×。
- 使用 compressed cyclotomic squaring 后，完整 pairing 相比对应的标量基线快 1.97×。

为什么不是 8×？

- 并行性往往只存在于更底层的 $\mathbb{F}_p$ 或 $\mathbb{F}_{p^2}$ 运算中。
- Tower field 中的模约减和向量布局转换会引入额外开销。
- Hybrid kernel 中包含效率较低的 (4×2) -way $\mathbb{F}_p$ 算术。
- `line_add` 在 $\mathbb{F}_{p^2}$ 层只能提供 2-way 并行。

结论：这是一个接近 2× 的 latency optimization，但其收益依赖 AVX-512IFMA，适用平台仍然有限。